ENGINEERING ANALYSIS

Native Movement Analysis

Evidence and experiments behind Latest coordinate and bounded movement.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Historical bounded speed-sensitive replacement - 7 September 2026

The former bounded native motion experiment used completed MediaPipe palm observations and separates resting noise from intentional velocity. It measures consecutive MediaPipe coordinates using capture timestamps and normalizes velocity by the player's directional reach. Calibration noise creates an elliptical X/Y resting region with hysteresis. Above it, one coherent two-dimensional newest-coordinate weight rises smoothly from `0.70` to `1.00` at 1.50 calibrated reach spans per second. Large travel and meaningful reversals are therefore direct. A stop settles inside the measured noise region on its first fresh result and exactly on the next, avoiding independent per-axis snaps.

The runtime cap is unconditionally `1.00`. Historical `motion_coordinate_boost` and `motion_coordinate_max` fields remain loadable, but cannot create extrapolation. No moving-average window, queue, replay, prediction, or core-side filter was added.

MediaPipe is the only live coordinate source because optical-flow movement was reported as jerky and unreliable even though the underlying MediaPipe tracking remained usable. The optical-flow implementation is retained as inactive source and historical trace support, but `motion_tracking` no longer enables it. The Dashboard no longer exposes the bounded curve; live movement uses direct Latest coordinates.

The initial MediaPipe-first deployment exposed two calibration mistakes in the bounded curve. Its `0.70` slow-follow weight was referenced to 60 Hz even though the Controller measured roughly 9-10 completed inferences per second; temporal normalization therefore raised most real samples to nearly one-to-one. The saved test calibration also contained saturated `1.0` jitter values, which the curve interpreted as a large image-space dead zone. Follow weighting now uses a 100 ms reference interval, and saturated jitter falls back to the small fixed floor while preserving the saved center, scale, and wrist.

`benchmark-vision-replay.py` version 2 retains timestamps and cue definitions in its aggregate local report. `benchmark-native-motion-curve.py` consumes that report, compares the former `.70 / 15 / 1.30` experiment, a capped error-driven reference, the actual speed curve, and direct latest coordinates, then evaluates 27 bounded candidates. It reports neutral span, selected-to-filtered error, medium 90% response, large first-sample misses, reversals, overshoot, continuity, and recognition age when present. The temporary physical clip is not currently available on the development Mac, so recorded-clip and synchronized live camera-to-display validation remain pending.

The 0.4.0 production choice is Latest coordinate. It publishes every valid reach-clamped MediaPipe point directly during continuous tracking. After a brief dropout, one result that contradicts established motion-or is unusually distant without strong forward alignment-is held until the next fresh result. Live tracing showed that allowing strongly aligned forward recovery immediately avoided the earlier compound catch-up jump while inserting no unnecessary holds in the tested gameplay window. This is a reacquisition guard, not smoothing or prediction.

The runtime now rejects malformed/non-finite landmark geometry, retains the calibration-compatible five-point wrist/knuckle average as the production anchor, and exports three alternative anchors for comparison. It clamps the selected point to player reach before processing. The historical comparison applied the same clamp to both Latest and Bounded lanes. Runtime clears history on stale/lost input and uses the frame capture timestamp rather than inference-start time for freshness.

The gameplay and dot recordings contain aggregate timings and sampled validity; they do not retain recognized, flow, selected, and filtered coordinates for each individual move. They cannot establish how many camera images or presented game frames contained intermediate hand positions. Receiver publications are not necessarily new measurements or different coordinates.

Observed evidence

30-second window	Valid Controller samples	Observed tracking losses
Earlier flow revision, dot movement	579/591 (98.0%)	6
Recognition fallback, fast dot movement	447/589 (75.9%)	34
Recognition fallback, actual gameplay	568/590 (96.3%)	3

A later MediaPipe-only, Dashboard-closed status run measured 96.5% detected samples, 65.4 ms median and 76.2 ms p95 source age, 52.2 ms median and 58.3 ms p95 inference, 2 ms p95 send work, and about 16 distinct native updates per second. This is a software-stage sample, not physical hand-to-display latency.

These were different physical movements, not controlled before/after trials. In gameplay, 395/590 polls reported `flow_unavailable`, 61 reported `flow_seed_unavailable`, and one reported `correction_budget`. Fallback therefore frequently delivered recognition coordinates rather than a newer flow estimate. These are sampled frame counts, not distinct recognition-result counts. The receiver observed 1,367 distinct valid publications in 30 seconds, not 1,367 new recognized positions or displayed images.

Isolated GPU feasibility result

The UNO Q exposes an Adreno 702 OpenGL ES 3.1 renderer. A custom ARM64 MediaPipe 0.10.18 research wheel initialized EGL and created the TensorFlow Lite GPU delegate, proving that the application container can reach the GPU when supplied with a compatible runtime. The synchronous Tasks Image graph measured roughly 664 ms warm p50 on GPU and 207 ms on CPU, compared with roughly 52 ms for the deployed MediaPipe Hands graph on live input. Repeated single-write tensor synchronization warnings accompanied the GPU run.

No gesture, reach, or smoothing threshold can remove hundreds of milliseconds inside the inference graph. The custom wheel and temporary runtime changes were removed from the Controller and are not release artifacts. The remaining useful experiment is a lean GPU palm-detection/landmark graph that keeps preprocessing on the GPU, prewarms once, returns only landmarks, and uses one newest result in flight. It must beat the proven CPU path by at least 20% without reducing recognition by more than one percentage point or worsening jitter or thermals.

Historical smoothing-only experiment

Before the speed-curve replacement, `scripts/analyze-motion-samples.py` exercised the error-driven native movement engine with an ideal instantaneous measured-position step. It assumed 60 updates/second, no input noise or loss, and the former code defaults (minimum .70, maximum 1.00, motion boost 4.00). The following table is retained as historical evidence for that implementation.

Step in normalized camera X	Time from first step sample to 95% of target
0.010	200 ms
0.025	200 ms
0.050	167 ms
0.100	Immediate on first sample

Step in normalized camera X	Time from first step sample to 95% of target
0.200	Immediate on first sample

With smoothing disabled, every modeled step passes through on its first sample. That comparison does **not** measure the jitter cost of disabling smoothing. The elapsed times exclude capture, recognition, transport, game simulation and display. They must not be added to independent timing percentiles. The final five-percent criterion includes a small settling tail; it is not a delay before movement begins.

The speed-dependent behavior explained the problem: the former coefficient used position error, then adjusted for elapsed time. It was adaptive, but was not a One Euro velocity-based filter. Large errors can bypass damping while small aiming corrections retain it. This is a plausible contributor to the reported difficulty catching and directing the ball, not proof of the complete cause.

The script now exercises the bounded speed curve and labels its parameters in new reports. Prediction accuracy cannot be tested from these aggregate reports. A predictor can extrapolate an old measurement, but abrupt stops and reversals can cause overshoot. Do not fit or enable prediction using these reports as ground truth.

Prepared trace extension

The optional finite diagnostic trace now includes, on each vision event:

- New recognition completion, its original capture timestamp, X/Y, confidence, and detection flag. In-flight frames contain no fabricated recognition event.
- Accepted anchor's original capture timestamp.
- Attempted optical-flow X/Y and a separate acceptance flag; a correction that exceeded its budget must not be counted as accepted flow.
- Selected normalized X/Y, fallback reason, and invalid-input reason.
- Final filtered normalized X/Y, smoothing parameters, and mapped native axes.

Capture sequence/timestamp and transport sequence remain available for correlation. Raw and filtered normalized coordinates share camera space; native axes use the player's reach mapping. Invalid selected coordinates are null. A repeated source is identifiable by anchor timestamp and must not be treated as a fresh velocity measurement. The existing trace remains opt-in, finite, buffered, and asynchronous. No video is recorded by this extension.

The instrumentation was subsequently deployed with the medium-jump trial below; trace collection has not yet been enabled. Before the next physical trial, enable a finite trace, then record the real hand and cabinet screen together with the iPhone. Test small corrections, a large move, a stop, and a reversal. Trace replay can compare smoothing outputs, while video supplies physical timing and overshoot evidence. Cross-device clocks require alignment; do not subtract them directly.

Reproduce the offline report with:

```
SH
python3 scripts/analyze-motion-samples.py \
  --samples-dir /tmp/uno-dot-baseline-x8heur1f \
  --output /tmp/new-motion-analysis.json
```

Analyze one finite per-frame trace without replaying controller input:

```sh
python3 scripts/analyze-motion-trace.py /tmp/controller.trace.json \
  --output /tmp/controller-motion-analysis.json
```

For a controlled directory whose filenames follow `min0.70-boost8.trace.json`, compare every configuration with:

```sh
python3 scripts/compare-motion-matrix.py /tmp/controller-motion-matrix \
  --output /tmp/controller-motion-matrix.json
```

Both tools create new reports rather than overwriting existing evidence. Their settling estimates describe normalized software coordinates, not physical hand-to-screen latency.

Medium-jump and bounded extrapolation trial

This historical experiment let `motion_coordinate_boost` override the boost in `update_native_motion`; null preserved the baseline value. The Controller trial used boost 15.0 instead of the baseline 4.0, with minimum .70. An additional experimental `motion_coordinate_max` cap of 1.30 allowed bounded extrapolation beyond the newest measured position. This was an error relative to the filtered position, not a speed threshold or a percentage of the game screen. It does not remove recognition age.

The actual-engine 60 Hz step model gives 0 ms additional settling time for .04 and .05 steps with boost 8, compared with 183 and 167 ms respectively at boost 4. A .005 step remains damped (200 ms to 95%, previously 217 ms). Noise and physical response still require the paired recording. Baseline synchronous tracking was unchanged. The bounded speed-sensitive replacement now ignores these legacy overrides during native movement while continuing to load them. Trace collection must be explicitly enabled for a physical test.

Six-configuration trace matrix

The matrix varied minimum smoothing and experimental motion boost while holding the 250 ms freshness limit, 320 px flow width, 25 ms correction budget, and maximum smoothing 1.00 fixed. Each window produced a finite trace with zero dropped records. The analysis uses `scripts/compare-motion-matrix.py`.

Minimum / boost	Vision events	Valid %	Losses	Age p95 (ms)	Medium settling p95 (ms)
0.70 / 4	834	69.18	14	104.5	0.0
0.70 / 8	751	56.72	17	175.0	0.0
0.70 / 12	895	90.17	7	87.2	84.6
0.85 / 4	884	89.37	4	104.1	114.0
0.85 / 8	886	97.74	1	85.4	96.9
0.85 / 12	897	97.44	2	83.1	63.8

These windows did not contain a controlled, repeated movement sequence, so the rows cannot select a production candidate. Validity and recognition age varied enough to confound settling comparisons. The results are a harness check and preliminary range, not proof that 0.85 / 12 is better. The live setting was restored to 0.70 / 8; calibration was unchanged.

Controlled three-window follow-up

The follow-up repeated the same center, small-correction, medium-jump, reversal, and center sequence for the baseline and two candidates. All three finite traces had zero dropped records.

Configuration	Vision events	Valid %	Losses	Age p95 (ms)	X error median / p95	Medium settling p95 (ms)
0.70 / 8 (baseline)	778	94.47	6	87.5	0.0011 / 0.0122	120.8
0.85 / 8	883	92.87	5	91.5	0.0006 / 0.0052	17.9
0.85 / 12	894	90.27	7	111.5	0.0004 / 0.0041	112.1

The controlled follow-up showed `0.85 / 8` was a useful intermediate smoothing candidate, but later gameplay testing found it too floaty. The live experiment was returned to minimum smoothing 0.70 and then raised to boost 15 with cap 1.30. Those later values are a user-driven exploratory setting, not a production default; repeat controlled traces before promoting them.